Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский Политехнический университет Петра Великого
—
Институт компьютерных наук и кибербезопасности

КУРСОВАЯ РАБОТА

«Разработка синтаксического анализатора языка Bash»
по дисциплине «Формальные грамматики и теория компиляторов»

Выполнил Чупров Е.А.

студент гр. 5131001/30501

Руководитель

ст. преподаватель Семьянов П.В.

Санкт-Петербург
2026

ОГЛАВЛЕНИЕ

1 ЦЕЛЬ РАБОТЫ **3**
2 ХОД РАБОТЫ 4
2.1 Теоретические сведения 4
2.2 Лексический анализатор языка 6
2.3 Синтаксический анализатор языка 8
2.4 Диагностические правила 20
2.5 Проверка конфликтов 21
3 РЕЗУЛЬТАТЫ **22**
4 ВЫВОД **25**
ПРИЛОЖЕНИЕ 1 **26**
ПРИЛОЖЕНИЕ 2 **29**

1 ЦЕЛЬ РАБОТЫ

Получение навыков проектирования грамматик языков программирования, самостоятельная разработка контекстно-свободной грамматики подмножества языка Bash и создание его синтаксического распознавателя с использованием генераторов лексических и синтаксических анализаторов flex и yacc.

Распознаватель решает только вопрос принадлежности текста языку. Сценарий он не исполняет и подстановки не раскрывает.

2 ХОД РАБОТЫ

2.1 Теоретические сведения

Bash — командный интерпретатор GNU и язык сценариев UNIX. По синтаксису он продолжает Bourne shell и раздел Shell Command Language стандарта POSIX, а собственные расширения Bash добавляет поверх этой основы. Сценарий представляет собой последовательность команд, разделённых точкой с запятой, переводом строки, амперсандом, конвейером или логическими операторами.

Весь Bash не помещается в одну контекстно-свободную грамматику. Here-документ читает текст до слова-ограничителя, арифметика и подстановки имеют собственные правила, алиасы, раскрытие тильды и раскрытие фигурных скобок меняют текст ещё до разбора. Поэтому в работе описано подмножество языка.

Подмножество устроено так. Файл начинается со строки #!/bin/bash; после пути допускаются только пробелы и табы. Файл из одного шебанга принимается, пустой файл — нет. Дальше идут команды с аргументами и перенаправлениями. Присваивание NAME=value может быть как отдельной командой, так и префиксом простой команды. Команды соединяются конвейером, операторами && и ||, точкой с запятой, амперсандом и переводом строки. Составными командами являются if, while, until, for, select, case, группа в фигурных скобках и подшелл в круглых скобках.

Проверка условия записывается как [...]. Слова вроде -lt, -f и = остаются обычными словами и не проверяются. У if есть ветки elif и else, у циклов — while, until, for и select, у case — ветки с образцами. Условие if, elif, while и until представляет собой один and_or: простую команду, проверку, конвейер или цепочку && и ||. Перед then и do нужен разделитель — точка с запятой или перевод строки. Функция записывается одним из трёх способов: имя() { ...; }, function имя { ...; } или function имя() { ...; }. Те же фигурные скобки могут быть отдельной командой.

Вне кавычек допускаются подстановки $имя, $1, $#, $@, $*, $$, $?, ${имя}, ${!имя}, ${имя[индекс]}, ${имя[@]}, ${имя[*]}, $(...) и $((...)). Внутри $(...) разбирается тот же список команд, что и в теле файла, но без шебанга. В кавычках знак доллара не разбирается: строка остаётся одним токеном.

2.2 Лексический анализатор языка

Лексический анализатор разбивает входной текст на последовательность лексем. Сканер начинает в состоянии INITIAL. В состояние BODY он переходит только после строки шебанга #!/bin/bash. Любой другой первый символ, лишний текст в строке шебанга и пустой файл дают ошибку на строке 1. Комментарии обрабатываются только в состоянии BODY: знак # начинает комментарий до конца строки, если стоит на границе токена.

Основные типы лексем приведены в таблице 1. Запись регулярных выражений здесь упрощена; точные шаблоны находятся в файле lex_bash.l.

Таблица 1 — Ключевые конструкции лексического анализатора

Категория	Конструкции и токены
Шебанг и комментарии	#!/bin/bash; комментарий от # до конца строки.
Ключевые слова	if, then, elif, else, fi, while, until, do, done, for, select, in, case, esac, function, local.
Имена	ID: латинская буква или _, далее латинские буквы, цифры или _.
Слова	WORD: непробельные байты без операторов, кавычек, обратной косой черты, обратного апострофа и $.
Присваивания	ASSIGN: NAME=..., включая +=, индексы [ ] и пустое значение.
Переменные и подстановки	DOLLAR_ID, DOLLAR_NUM, DOLLAR_SPECIAL, DOLLAR_BRACE, DOLLAR_LPAREN, ARITH; ${...} с !, индексом, @ или *.

Строковые литералы	'...' и "..." — один токен STRING.
Операторы и разделители	&&, ||, ;;, ;, &, |, <, >, >>, 2>, 2>&1, (,), {, }, [,], NL.

Flex выбирает самое длинное совпадение. Поэтому >> не распадается на два знака >, а 2>&1 не распадается на 2> и отдельный фрагмент. Порядок правил решает только спор одинаковой длины, например между ключевым словом и именем. Пробел и таб токенов не порождают. Перевод строки в теле программы становится токеном NL.

Строки читаются отдельно от основного автомата. В одинарных кавычках обратная косая черта является обычным байтом, а перевод строки входит в строку. В двойных кавычках обратная косая черта прячет следующий символ, в том числе перевод строки; кавычка и обратная косая черта могут быть экранированы. Внутри кавычек подстановки не разбираются. Если закрывающей кавычки нет до конца файла, выводится сообщение о незакрытой строке с номером строки открывающей кавычки.

Подстановка $((...)) читается как одна лексема ARITH. Внутри допускаются латинские буквы, цифры, пробелы и знаки +, -, *, /, %, <, >, =, !, &, | и ^. Таба, перевода строки, знака доллара, кавычек и вложенных круглых скобок внутри ARITH нет. Формула не проверяется и не вычисляется: подходят и $((count + 1)), и $((1+)). Если внутри встречается недопустимый байт, текст читается как знак $, открывающая скобка и дальнейший подшелл, а не как ARITH.

Полный код анализатора приведён в **Приложении 1**.

2.3 Синтаксический анализатор языка

Стартовый символ грамматики — program. Иерархия правил сверху вниз выглядит так: program, list, stmt_seq, and_or, pipeline, command. В правилах, которые описывают допустимый язык, семантических вычислений нет: анализатор не выполняет команды и не раскрывает подстановки. Семантические действия есть только у ошибочных псевдоправил, которые печатают диагностику и завершают разбор. Основные синтаксические элементы перечислены в таблице 2.

Таблица 2 — Основные синтаксические элементы

Нетерминал	**Описание**
program	Шебанг и список команд.
list	Список команд; может быть пустым.
stmt_seq	Команды and_or подряд.
separator	;, & или NL.
nl_opt	Ноль или больше NL.
and_or	Конвейеры через && и ||.
pipeline	Команды через |.
command	Простая, тест, составная, функция, local, ошибочные then/do.
simple_command	Присваивания, имя, аргументы, редиректы.
assign_part	ASSIGN или ASSIGN со скобками.
assign_list	Одно или несколько assign_part.
test_command	[...] и редиректы.
compound_command	Составная команда и редиректы.
if_clause, while_clause, until_clause, for_clause, select_clause, case_clause	Составные команды.

function_def, brace_group, subshell, local_stmt	Функция, группа, подшелл, local.
cmdsubst	$(list).
sep	; или NL перед then/do.
redir_list, redir, redir_target	Редиректы и их цели.

Программа обязана начинаться с шебанга. За ним идёт список команд. Файл из одного шебанга принадлежит языку, пустой файл — нет. Разделитель separator завершает команду: точка с запятой и перевод строки равносильны, амперсанд помечает фон. Правило nl_opt разрешает пустые строки и перенос после &&, || и | без пустой команды.

```
37  program:
38      SHEBANG list
39      ;
40
41  list:
42      nl_opt
43      | nl_opt stmt_seq
44      | nl_opt stmt_seq separator
45      ;
46
47  /* последовательность команд */
48  stmt_seq:
49      and_or
50      | stmt_seq separator and_or
51      ;
52
53  separator:
54      SEMI nl_opt
55      | AMP nl_opt
56      | NL nl_opt
57      ;
58
59  nl_opt:
60      | nl_opt NL
61      ;
```

Рисунок 1 – Начало грамматики

Правило and_or строит цепочку конвейеров через && и ||. Правило pipeline строит цепочку команд через |. После &&, || и | может стоять пустая строка: её забирает nl_opt. Конвейер стоит ниже списка, поэтому в записи a && b | c сначала связывается b | c, и только потом результат соединяется с a через &&. Одно звено такой цепочки — command: простая команда, проверка [...], составная команда, функция, local или ошибочное ключевое слово. Составная команда — это if, while, until, for, select, case, группа или подшелл, и после неё может стоять перенаправление. Правило stray_keyword ловит then и do не на своём месте.

```
 63  and_or:
 64      pipeline
 65      | and_or AND nl_opt pipeline
 66      | and_or OR nl_opt pipeline
 67      ;
 68
 69  /* конвейер | */
 70  pipeline:
 71      command
 72      | pipeline PIPE nl_opt command
 73      ;
 74
 75  command:
 76      simple_command
 77      | test_command
 78      | compound_command
 79      | function_def redir_list
 80      | local_stmt
 81      | stray_keyword
 82      ;
100  compound_command:
101      compound_head redir_list
102      ;
124  compound_head:
125      if_clause
126      | while_clause
127      | until_clause
128      | for_clause
129      | select_clause
130      | case_clause
131      | brace_group
132      | subshell
133      ;
134
135  /* ( команды ) */
136  subshell:
137      LPAREN list RPAREN
138      ;
```

Рисунок 2 – Списки && и ||, конвейер и составная команда

Простая команда начинается с одного или нескольких assign_part. Каждый assign_part — это либо токен ASSIGN, либо ASSIGN со списком аргументов в круглых скобках. Такая запись покрывает FOO=, FOO=1, arr+=, arr+=x, arr=(a b c), arr[0]=1 и arr[i]=value. Если после префикса есть слово, оно считается именем команды: FOO=1 echo x или arr=(a b) echo x. Если слова нет, вся запись является присваиванием. Аргументом простой команды может быть и assign_part, поэтому echo FOO=1 и declare -a arr=(1 2 3) остаются обычными командами.

```
 84  simple_command:
 85      assign_list
 86      | assign_list word arg_list redir_list
 87      | word arg_list redir_list
 88      ;
104  assign_part:
105      ASSIGN
106      | ASSIGN LPAREN arg_list RPAREN
107      ;
108
109  assign_list:
110      assign_part
111      | assign_list assign_part
112      ;
113
114  arg_list:
115      | arg_list arg
116      ;
117
118  arg:
119      word
120      | assign_part
121      | LBRACE WORD RBRACE
122      ;
```

Рисунок 3 – Присваивание и простая команда

Имя команды — то же слово, что и аргумент: ID, WORD, строка или подстановка. Слово word включает ID, WORD, STRING и expansion. Expansion включает DOLLAR_ID, DOLLAR_NUM, DOLLAR_SPECIAL, DOLLAR_BRACE, ARITH и cmdsubst. Внутри cmdsubst разбирается тот же список команд, что и в теле файла.

```
140  word:
141      ID
142      | WORD
143      | STRING
144      | expansion
145      ;
146
147  /* $имя, $1, ${имя}, $# $@ $* $$ $? */
148  expansion:
149      DOLLAR_ID
150      | DOLLAR_NUM
151      | DOLLAR_SPECIAL
152      | DOLLAR_BRACE
153      | cmdsubst
154      | ARITH
155      ;
156
157  /* $(...) */
158  cmdsubst:
159      DOLLAR_LPAREN list RPAREN
160      ;
```

Рисунок 4 – Слово и подстановки

В правиле arg, кроме word и assign_part, есть альтернатива из открывающей фигурной скобки, одного WORD и закрывающей фигурной скобки. Поэтому в позиции аргумента принимается ограниченная запись вида {a,b} или {1..100}. Это не полноценное раскрытие фигурных скобок: содержимое не раскрывается, а запись {a} не принимается, потому что a

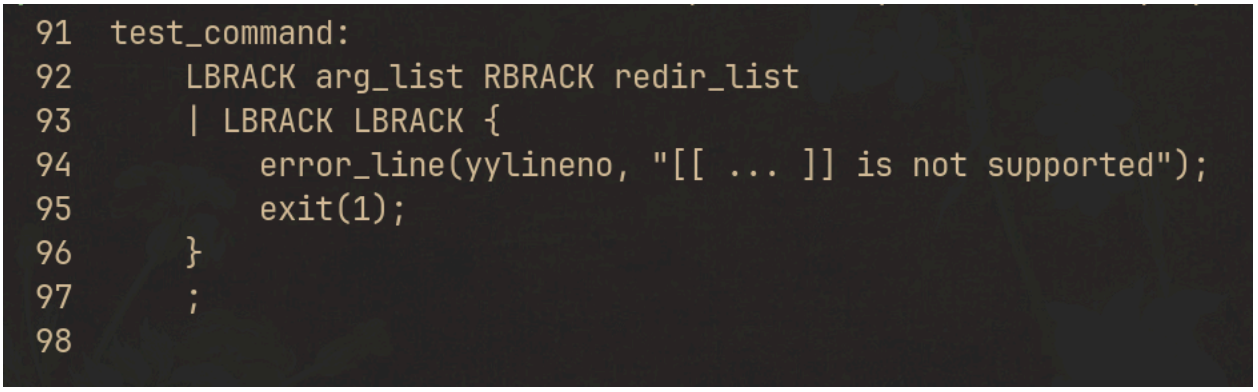
распознаётся как ID, а не как WORD. Такая особенность возникла в замороженной реализации и подробнее описана в разделе 2.5.

Перенаправлений может не быть. Допустимы >, >>, <, 2> и 2>&1. Они стоят в конце простой команды, проверки, составной команды и объявления функции. Цель >, >>, < и 2> — слово: имя, обычное слово, строка или подстановка. Конструкция FOO=1 целью не бывает. Два знака < подряд означают here-документ; такое правило отвергает запись и печатает here-document is not supported. Отдельного токена 2>> нет, поэтому запись 2>> не собирается в одно перенаправление.

```
162  redir_list:
163      | redir_list redir
164      ;
165
166  /* > >> < 2> 2>&1; << нельзя here-документ */
167  redir:
168      GT redir_target
169      | DGREAT redir_target
170      | LT redir_target
171      | LT LT {
172          error_line(yylineno, "here-document is not supported");
173          exit(1);
174      }
175      | REDIR_ERR redir_target
176      | REDIR_ERR_OUT
177      ;
178
179  redir_target:
180      word
181      ;
```

Рисунок 5 – Перенаправления

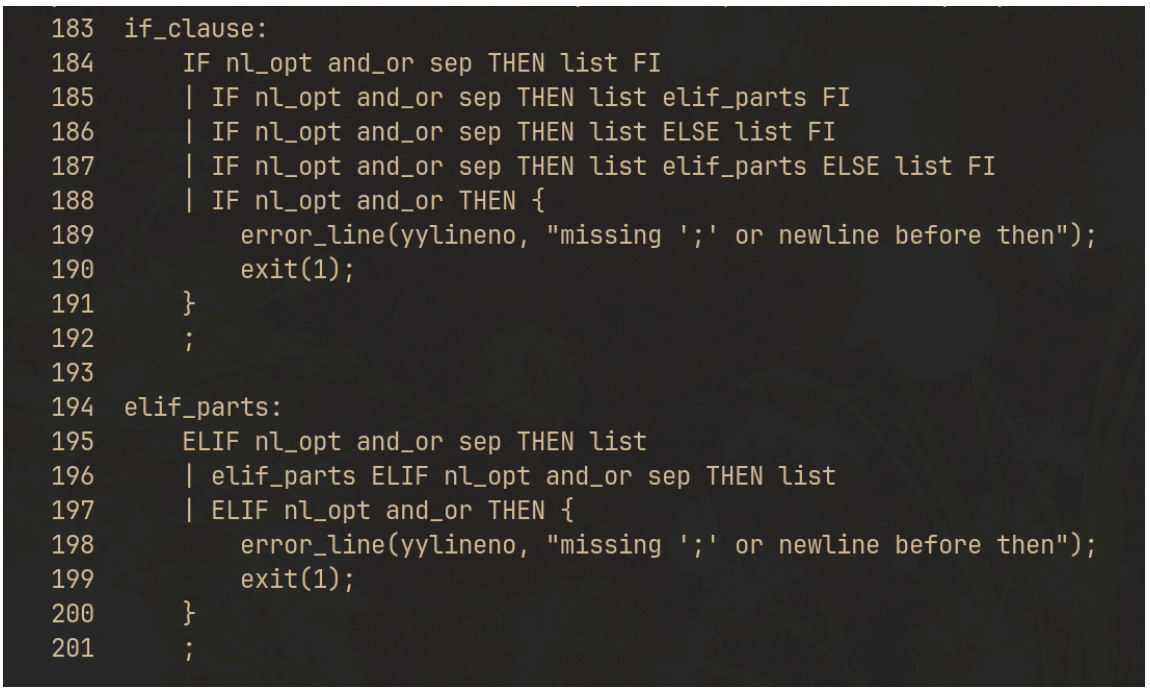
Проверка [...] устроена как список аргументов между открывающей и закрывающей квадратными скобками. Слов может не быть, как в [ ]. Знаки -f, = и -lt остаются обычными словами. Две открывающие скобки подряд, в том числе разделённые пробелом, дают сообщение [[...]] is not supported. После закрывающей скобки допускается перенаправление.



```
91  test_command:
92      LBRACK arg_list RBRACK redir_list
93      | LBRACK LBRACK {
94          error_line(yylineno, "[[ ... ]] is not supported");
95          exit(1);
96      }
97      ;
98
```

Рисунок 6 – Проверка в квадратных скобках

У if условие — один and_or, затем sep, затем then, список команд и fi. Веток elif может быть несколько, ветка else необязательна. Тело после then может быть пустым, пустые строки после then и elif разрешены. Если перед then нет разделителя, печатается missing ';' or newline before then. Две команды через точку с запятой одним условием не считаются: первая точка с запятой уже закрывает условие. Ветку if нельзя закрыть словом done — это обычная синтаксическая ошибка без пояснения.



```
183  if_clause:
184      IF nl_opt and_or sep THEN list FI
185      | IF nl_opt and_or sep THEN list elif_parts FI
186      | IF nl_opt and_or sep THEN list ELSE list FI
187      | IF nl_opt and_or sep THEN list elif_parts ELSE list FI
188      | IF nl_opt and_or THEN {
189          error_line(yylineno, "missing ';' or newline before then");
190          exit(1);
191      }
192      ;
193
194  elif_parts:
195      ELIF nl_opt and_or sep THEN list
196      | elif_parts ELIF nl_opt and_or sep THEN list
197      | ELIF nl_opt and_or THEN {
198          error_line(yylineno, "missing ';' or newline before then");
199          exit(1);
200      }
201      ;
```

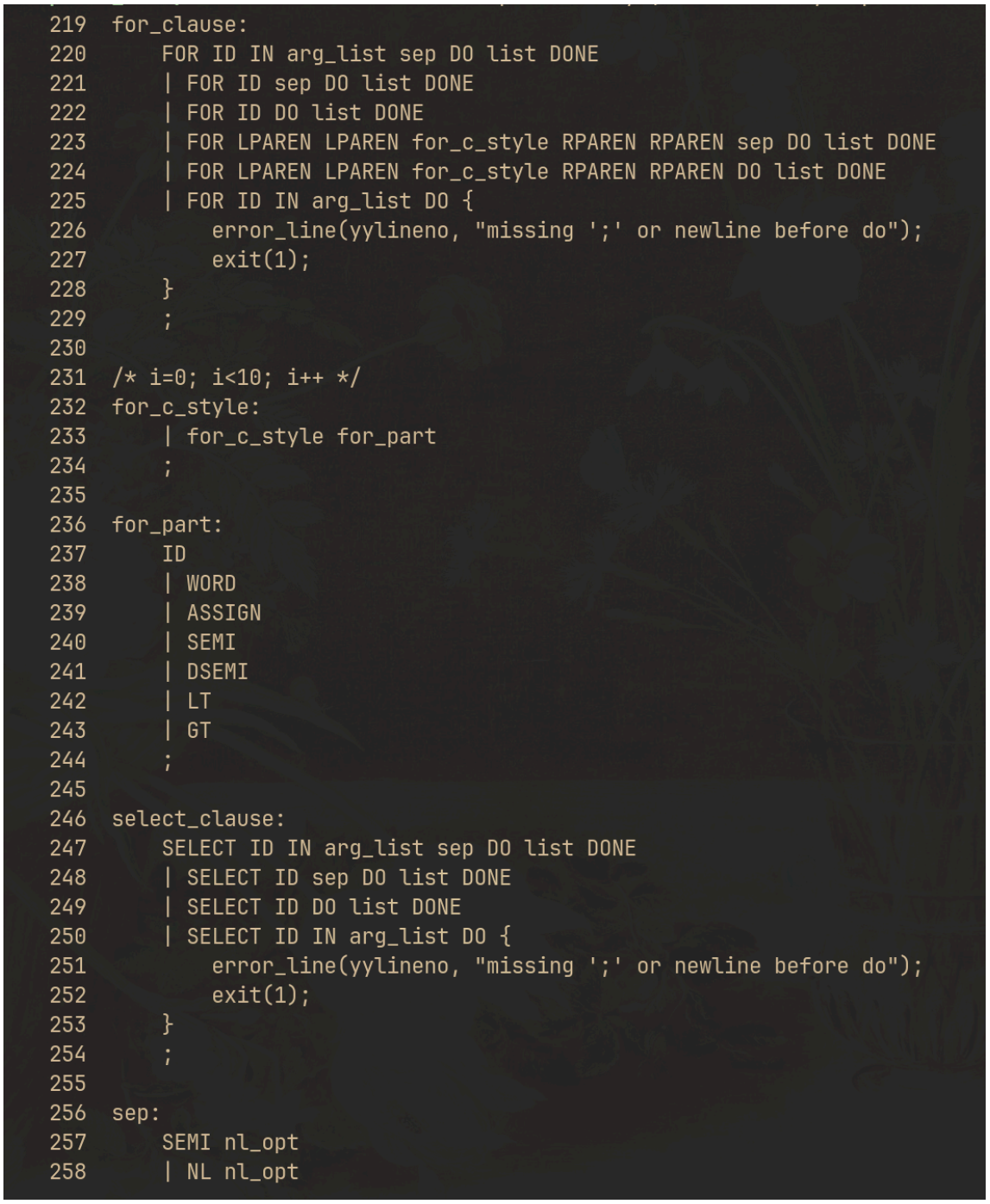
Рисунок 7 – Условие if и elif

У while и until та же схема: условие and_or, разделитель sep, слово do, тело и done. Перед do нужен разделитель; без него печатается missing ';' or newline before do. Тело может быть пустым.

```
203  while_clause:
204      WHILE nl_opt and_or sep DO list DONE
205      | WHILE nl_opt and_or DO {
206          error_line(yylineno, "missing ';' or newline before do");
207          exit(1);
208      }
209      ;
210
211  until_clause:
212      UNTIL nl_opt and_or sep DO list DONE
213      | UNTIL nl_opt and_or DO {
214          error_line(yylineno, "missing ';' or newline before do");
215          exit(1);
216      }
217      ;
```

Рисунок 8 – Циклы while и until

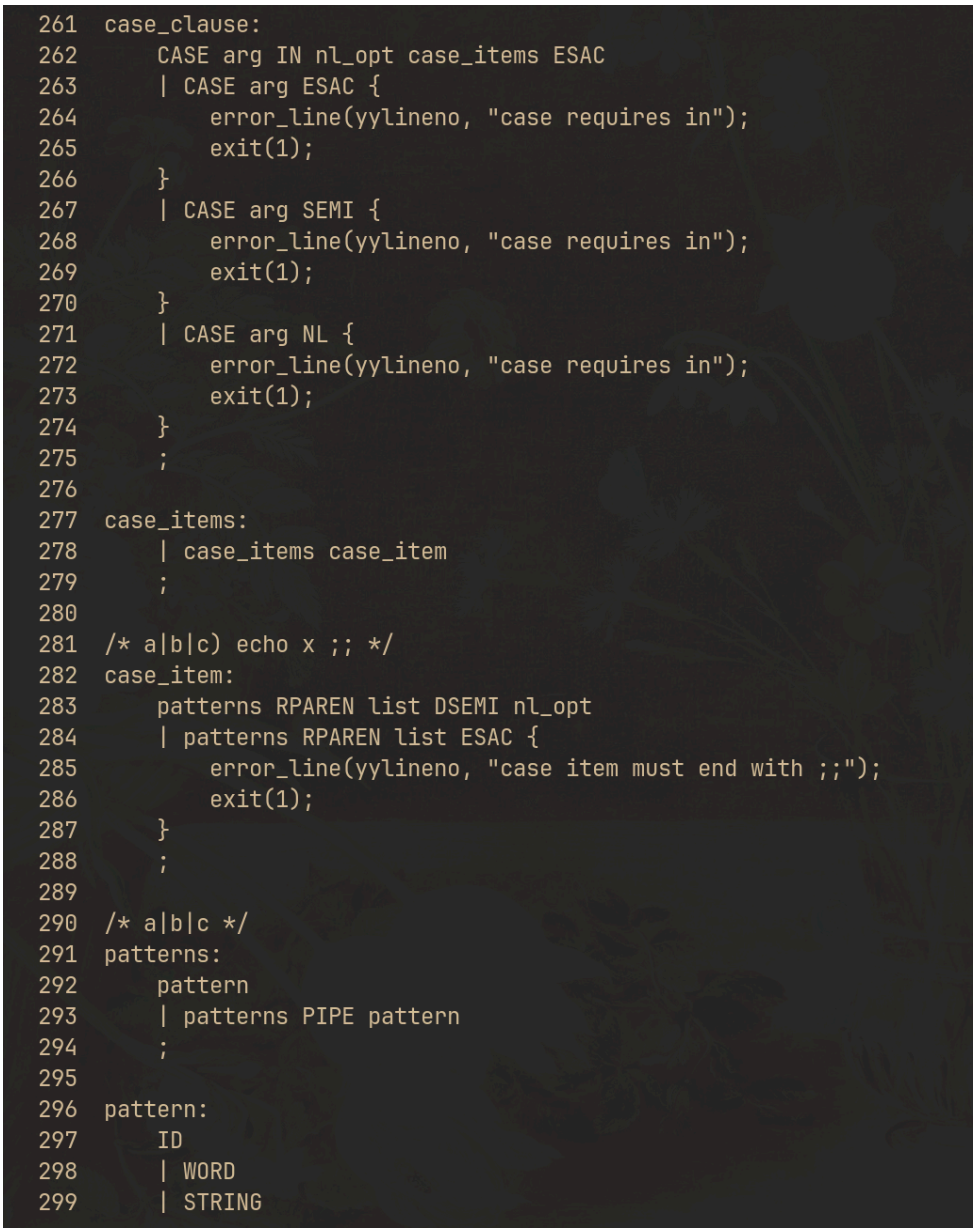
У for и select есть форма со списком in, форма без in и форма без разделителя перед do. Запись for name in a b; do ... done, for name; do ... done и for name do ... done допустимы; у select формы те же. Имя цикла — только ID. Список после in может быть пустым. У for дополнительно есть C-подобная форма for ((...)); do ... done и for ((...)) do ... done. Между двойными скобками допускаются ID, WORD, ASSIGN, точка с запятой, двойная точка с запятой, < и >. Перевод строки между for и ((не принимается; && и || внутри заголовка не входят. Внутри заголовка for ((...)) по-прежнему действуют обычные лексемы, а не отдельная арифметическая грамматика.



```
219 for_clause:
220     FOR ID IN arg_list sep DO list DONE
221     | FOR ID sep DO list DONE
222     | FOR ID DO list DONE
223     | FOR LPAREN LPAREN for_c_style RPAREN RPAREN sep DO list DONE
224     | FOR LPAREN LPAREN for_c_style RPAREN RPAREN DO list DONE
225     | FOR ID IN arg_list DO {
226         error_line(yylineno, "missing ';' or newline before do");
227         exit(1);
228     }
229     ;
230
231 /* i=0; i<10; i++ */
232 for_c_style:
233     | for_c_style for_part
234     ;
235
236 for_part:
237     ID
238     | WORD
239     | ASSIGN
240     | SEMI
241     | DSEMI
242     | LT
243     | GT
244     ;
245
246 select_clause:
247     SELECT ID IN arg_list sep DO list DONE
248     | SELECT ID sep DO list DONE
249     | SELECT ID DO list DONE
250     | SELECT ID IN arg_list DO {
251         error_line(yylineno, "missing ';' or newline before do");
252         exit(1);
253     }
254     ;
255
256 sep:
257     SEMI nl_opt
258     | NL nl_opt
```

Рисунок 9 – Циклы for и select

У case слово стоит после case, затем in, затем ноль или больше веток и esac. Каждый образец — ID, WORD или строка; несколько образцов в одной ветке разделяются |. Каждая ветка, включая последнюю, заканчивается ;;. Если после слова нет in, печатается case requires in; если ветка дошла до esac без ;;, печатается case item must end with ;;. После in пустые строки разрешены.



```
261  case_clause:
262      CASE arg IN nl_opt case_items ESAC
263      | CASE arg ESAC {
264          error_line(yylineno, "case requires in");
265          exit(1);
266      }
267      | CASE arg SEMI {
268          error_line(yylineno, "case requires in");
269          exit(1);
270      }
271      | CASE arg NL {
272          error_line(yylineno, "case requires in");
273          exit(1);
274      }
275      ;
276
277  case_items:
278      | case_items case_item
279      ;
280
281  /* a|b|c) echo x ;; */
282  case_item:
283      patterns RPAREN list DSEMI nl_opt
284      | patterns RPAREN list ESAC {
285          error_line(yylineno, "case item must end with ;;");
286          exit(1);
287      }
288      ;
289
290  /* a|b|c */
291  patterns:
292      pattern
293      | patterns PIPE pattern
294      ;
295
296  pattern:
297      ID
298      | WORD
299      | STRING
```

Рисунок 10 – Образцы case

Функция задаётся тремя способами: имя() { ...; }, function имя { ...; } и function имя() { ...; }. Тело — та же группа в фигурных скобках, что и отдельная команда. Перед закрывающей скобкой в непустой группе нужен разделитель: точка с запятой, перевод строки или амперсанд. Поэтому запись { echo x & } принимается, а { echo x } даёт missing ';' or newline before '}'. Пустые { } и ( ) допустимы. Подшелл — список команд в круглых скобках; запись ((echo z)) читается как два вложенных подшелла, а не как арифметика.

```
function_def:
    ID LPAREN RPAREN nl_opt brace_group
    | FUNCTION ID nl_opt brace_group
    | FUNCTION ID LPAREN RPAREN nl_opt brace_group
    ;

/* { }; перед } нужен ; или перевод строки */
brace_group:
    LBRACE nl_opt RBRACE
    | LBRACE nl_opt stmt_seq separator RBRACE
    | LBRACE nl_opt stmt_seq RBRACE {
        error_line(yylineno, "missing ';' or newline before '}'");
        exit(1);
    }
    ;
```

Рисунок 11 – Функция и её тело

У local должен быть хотя бы один аргумент. Аргумент — ID, WORD, ASSIGN, ASSIGN со строкой или подстановкой, либо ASSIGN со списком в круглых скобках. Запись local a= b — два аргумента: пустое присваивание a= и имя b. Запись local a=b — один токен ASSIGN. Перенаправление после local не предусмотрено. Стоит ли local внутри функции, грамматика не проверяет: local на верхнем уровне текста тоже принимается.

```
local_stmt:
    LOCAL local_names
    ;

local_names:
    local_item
    | local_names local_item
    ;

local_item:
    ID
    | WORD
    | ASSIGN
    | ASSIGN local_value
    | ASSIGN LPAREN arg_list RPAREN
    ;

local_value:
    STRING
    | expansion
```

Рисунок 12 – Объявление local

Ключевые слова then и do на месте команды дают сообщения unexpected then и unexpected do. Слова fi, done, else, elif и esac так не помечены: они закрывают тело, а пустое тело неотличимо от команды из одного такого слова. Поэтому одиночное fi отвергается, но без отдельного пояснения.

2.4 Диагностические правила

Обычная синтаксическая ошибка даёт только номер строки, потому что bison не знает, что именно пошло не так. Чтобы для типичных ошибок выдавать пояснение, в грамматику добавлены диагностические правила, или псевдоправила. Такое правило описывает не допустимую конструкцию, а типичную ошибочную запись. Например, правило IF and_or THEN срабатывает, когда после условия if сразу идёт then, без ; или перевода строки. Остальные псевдоправила приведены в таблице 3.

Таблица 3 — Диагностические правила

Правило	**Ошибочная запись**	**Сообщение**
redir: LT LT	cat <<EOF	here-document is not supported
test_command: LBRACK LBRACK	[[-f x]]	[[...]] is not supported
if_clause: IF and_or THEN; elif_parts: ELIF and_or THEN	if true then	missing ';' or newline before then
while_clause, until_clause, for_clause, select_clause	while true do; for i in a b do	missing ';' or newline before do
case_clause: CASE arg ESAC / SEMI / NL	case $x esac	case requires in
case_item: patterns RPAREN list ESAC	a) echo ok esac	case item must end with ;;
brace_group: LBRACE nl_opt stmt_seq RBRACE	f() { echo a }	missing ';' or newline before '}'
stray_keyword: THEN, DO	then или do не на месте	unexpected then, unexpected do

2.5 Проверка конфликтов

Сборка bison не сообщает конфликтов сдвига и свёртки. Конфликтов двух свёрток также нет. В файле y.output число конфликтов равно нулю. Это подтверждает рисунок 13.

```
compiler_labs/course main ● ? ❱ cat compil.sh
flex lex_bash.l         # Генерирует lex.yy.c
echo "lex"
bison -d -y -v pars_bash.y    # Генерирует y.tab.c и y.tab.h
echo "bison"
gcc -o bash_parser y.tab.c lex.yy.c -lfl  # -lfl для Flex библиотеки
echo "gcc"
compiler_labs/course main ● ? ❱ ./compil.sh
lex
bison
gcc
```

Рисунок 13 – Сборка без конфликтов

3 РЕЗУЛЬТАТЫ

Текст принадлежит языку: stdout пуст, stderr пуст, код возврата 0. Текст не принадлежит языку: stdout пуст, stderr содержит одну строку Error: line N или Error: line N: пояснение, код возврата 1.

Номер строки считается с единицы. После первой ошибки разбор не продолжается. Для токена перевода строки flex увеличивает счётчик при чтении символа, поэтому ошибка на таком токене может относиться к следующей строке. Полный список пояснений приведён в таблице 4.

Таблица 4 — Пояснения к номеру строки

Пояснение	Когда печатается
missing shebang	Нет #!/bin/bash или файл пуст
extra text in shebang	Лишний текст после #!/bin/bash
here-document is not supported	<<
[[...]] is not supported	Две скобки [[
missing ';' or newline before then	Нет ; или NL перед then
missing ';' or newline before do	Нет ; или NL перед do
missing ';' or newline before '}'	Нет ;, & или NL перед }
unexpected then	then не на месте
unexpected do	do не на месте
case requires in	case без in
case item must end with ;;	Ветка без ;; перед esac
unterminated single quote	Нет закрывающей '
unterminated double quote	Нет закрывающей "
unexpected character	Байт без лексемы, в том числе \ и обратный апостроф вне кавычек

Примеры ниже показывают типичные принимаемые и отвергаемые записи.

На рисунках 14 и 15 показаны принимаемые сценарии: if с elif и else, а также функция с local. Сообщений нет, код возврата равен нулю. На рисунках 16, 17 и 18 показаны отказы: отсутствие шебанга, here-документ и синтаксическая ошибка на строке 4. На рисунке 19 приведён большой принимаемый сценарий.

```
compiler_labs/course main ● ? ❱ cat tests/accept/12-if-elif-else.sh
#!/bin/bash
if [ "$1" = a ]; then
  echo a
elif [ "$1" = b ]; then
  echo b
else
  echo other
fi

compiler_labs/course main ● ? ❱ ./bash_parser tests/accept/12-if-elif-else.sh
```

Рисунок 14 – Принимаемый сценарий: if, elif и else

```
compiler_labs/course main ● ? ❱ cat tests/accept/19-local.sh
#!/bin/bash
f() {
  local name
  local name="$1"
  local a b=1
}

compiler_labs/course main ● ? ❱ ./bash_parser tests/accept/19-local.sh
```

Рисунок 15 – Принимаемый сценарий: функция и local

```
compiler_labs/course main ● ? ❱ cat tests/reject/no-shebang.sh
echo hello

compiler_labs/course main ● ? ❱ ./bash_parser tests/reject/no-shebang.sh
Error: line 1: missing shebang
```

Рисунок 16 – Отказ: нет шебанга

```
compiler_labs/course main ● ? ❱ cat tests/reject/here-doc.sh
#!/bin/bash
cat <<EOF
x
EOF

compiler_labs/course main ● ? ❱ ./bash_parser tests/reject/here-doc.sh
Error: line 2: here-document is not supported
```

Рисунок 17 – Отказ: here-документ не поддерживается

```
compiler_labs/course main ● ? ❱ cat tests/reject/if-wrong-close.sh
#!/bin/bash
if [ a ]; then
  echo x
done

compiler_labs/course main ● ? ❱ ./bash_parser tests/reject/if-wrong-close.sh
Error: line 4
```

Рисунок 18 – Отказ: ошибка на строке 4

```
compiler_labs/course main ● ? ❱ cat tests/accept/30-overview.sh
#!/bin/bash
name="World"
count=0
if [ $# -lt 1 ]; then echo usage; fi
echo $name | wc -l > /tmp/out
greet() {
  local name="$1"
  echo ${name}
}
for item in a b; do greet $item; done
case $item in
a|b) echo ok ;;
*) echo other ;;
esac

compiler_labs/course main ● ? ❱ ./bash_parser tests/accept/30-overview.sh
```

Рисунок 19 – Принимаемый сценарий: большой файл

4 ВЫВОД

В ходе работы для подмножества Bash разработана собственная контекстно-свободная грамматика, на flex и yacc собран синтаксический распознаватель; сборка грамматики проходит без конфликтов. Корректный текст принимается без сообщений с кодом 0, при первой ошибке выводится номер строки и код 1. Полный Bash не укладывается в одну контекстно-свободную грамматику, поэтому here-документ, арифметика и сложные подстановки оставлены за границей подмножества.

ПРИЛОЖЕНИЕ 1

```
%{
#include <stdio.h>
#include <stdlib.h>
#include "y.tab.h"

extern void error_line(int line, const char *msg);

static int read_single_quote(void);
static int read_double_quote(void);
%}

%option yylineno noyywrap
%x BODY

ID      [A-Za-z_][A-Za-z0-9_]*
NUM     [0-9]+
WORD    [^ \t\n|&;<>(){}[\]"'`$\\]+

%%
"#!/bin/bash"[ \t]*\n  { BEGIN(BODY); return SHEBANG; }
"#!/bin/bash"[ \t]*    { BEGIN(BODY); return SHEBANG; }
"#!/bin/bash".*        { error_line(1, "extra text in shebang"); exit(1); }
.                      { error_line(1, "missing shebang"); exit(1); }
\n                     { error_line(1, "missing shebang"); exit(1); }
<INITIAL><<EOF>>       { error_line(1, "missing shebang"); exit(1); }

<BODY>[ \t]+                    { }
<BODY>#.*                       { }

<BODY>"if"                      { return IF; }
<BODY>"then"                    { return THEN; }
<BODY>"elif"                    { return ELIF; }
<BODY>"else"                    { return ELSE; }
<BODY>"fi"                      { return FI; }
<BODY>"while"                   { return WHILE; }
<BODY>"until"                   { return UNTIL; }
<BODY>"do"                      { return DO; }
<BODY>"done"                    { return DONE; }
<BODY>"for"                     { return FOR; }
<BODY>"select"                  { return SELECT; }
<BODY>"in"                      { return IN; }
<BODY>"case"                    { return CASE; }
<BODY>"esac"                    { return ESAC; }
<BODY>"function"                { return FUNCTION; }
<BODY>"local"                   { return LOCAL; }

<BODY>"2>&1"                    { return REDIR_ERR_OUT; }
<BODY>"2>"                      { return REDIR_ERR; }
<BODY>">>"                      { return DGREAT; }
<BODY>"&&"                      { return AND; }
<BODY>"||"                      { return OR; }
<BODY>";;"                      { return DSEMI; }

<BODY>"|"                       { return PIPE; }
```

```
        <BODY>"&"                       { return AMP; }
        <BODY>";"                       { return SEMI; }
        <BODY>"<"                       { return LT; }
        <BODY>">"                       { return GT; }
        <BODY>"("                       { return LPAREN; }
        <BODY>")"                       { return RPAREN; }
        <BODY>"{"                       { return LBRACE; }
        <BODY>"}"                       { return RBRACE; }
        <BODY>"["                       { return LBRACK; }
        <BODY>"]"                       { return RBRACK; }

        <BODY>"$(("[0-9A-Za-z+\-*/%<>=!&|^ ]+"))"  { return ARITH; }

        <BODY>"$("                      { return DOLLAR_LPAREN; }
        <BODY>"${"{ID}"}"               { return DOLLAR_BRACE; }
        <BODY>"${!"{ID}"}"              { return DOLLAR_BRACE; }
        <BODY>"${"{ID}"["{NUM}"]}"      { return DOLLAR_BRACE; }
        <BODY>"${"{ID}"["{ID}"]}"       { return DOLLAR_BRACE; }
        <BODY>"${"{ID}"["[@*]"]}"       { return DOLLAR_BRACE; }
        <BODY>"${"                      { error_line(yylineno, NULL); exit(1); }
        <BODY>"#"|"@"|"* "|""|"?"  { return DOLLAR_SPECIAL; }
        <BODY>"$"{ID}                   { return DOLLAR_ID; }
        <BODY>"$"[0-9]+                 { return DOLLAR_NUM; }
        <BODY>"$"                       { error_line(yylineno, NULL); exit(1); }

        <BODY>{ID}\+={WORD}             { return ASSIGN; }
        <BODY>{ID}\+=                   { return ASSIGN; }
        <BODY>{ID}
                                                ID
        ={WORD}       { return ASSIGN; }
        <BODY>{ID}
                                                NUM
        ={WORD}      { return ASSIGN; }
        <BODY>{ID}
                                                ID
        =              { return ASSIGN; }
        <BODY>{ID}
                                                NUM
        =            { return ASSIGN; }
        <BODY>{ID}={WORD}               { return ASSIGN; }
        <BODY>{ID}=                     { return ASSIGN; }
        <BODY>{ID}                      { return ID; }
        <BODY>{WORD}                    { return WORD; }

        <BODY>'                         { return read_single_quote(); }
        <BODY>\"                        { return read_double_quote(); }

        <BODY>\n                        { return NL; }
        <BODY>.                                 { error_line(yylineno, "unexpected character");
exit(1); }

        %%

        static int read_single_quote(void)
        {
            int start_line = yylineno;
```

```
    int c;

    while ((c = input()) != 0) {
        if (c == '\'')
            return STRING;
    }

    error_line(start_line, "unterminated single quote");
    exit(1);
}

static int read_double_quote(void)
{
    int start_line = yylineno;
    int c;
    int escaped = 0;

    while ((c = input()) != 0) {
        if (escaped) {
            escaped = 0;
            continue;
        }
        if (c == '\\') {
            escaped = 1;
            continue;
        }
        if (c == '"')
            return STRING;
    }

    error_line(start_line, "unterminated double quote");
    exit(1);
}
```

ПРИЛОЖЕНИЕ 2

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex(void);
extern int yylineno;
extern FILE *yyin;

void error_line(int line, const char *msg)
{
    if (msg)
        fprintf(stderr, "Error: line %d: %s\n", line, msg);
    else
        fprintf(stderr, "Error: line %d\n", line);
}

void yyerror(char *msg)
{
    (void)msg;
    error_line(yylineno, NULL);
}
%}

%token SHEBANG
%token IF THEN ELIF ELSE FI
%token WHILE UNTIL DO DONE
%token FOR SELECT IN CASE ESAC FUNCTION LOCAL
%token ID WORD STRING ASSIGN
%token DOLLAR_ID DOLLAR_NUM DOLLAR_SPECIAL DOLLAR_BRACE DOLLAR_LPAREN ARITH
%token LBRACK RBRACK LBRACE RBRACE LPAREN RPAREN
%token PIPE AND OR AMP SEMI DSEMI NL
%token LT GT DGREAT REDIR_ERR REDIR_ERR_OUT

%start program

%%
program:
    SHEBANG list
    ;

list:
    nl_opt
    | nl_opt stmt_seq
    | nl_opt stmt_seq separator
    ;

/* последовательность команд */
stmt_seq:
    and_or
    | stmt_seq separator and_or
    ;

separator:
```

```
    SEMI nl_opt
    | AMP nl_opt
    | NL nl_opt
    ;

nl_opt:
    | nl_opt NL
    ;

and_or:
    pipeline
    | and_or AND nl_opt pipeline
    | and_or OR nl_opt pipeline
    ;

/* конвейер | */
pipeline:
    command
    | pipeline PIPE nl_opt command
    ;

command:
    simple_command
    | test_command
    | compound_command
    | function_def redir_list
    | local_stmt
    | stray_keyword
    ;

simple_command:
    assign_list
    | assign_list word arg_list redir_list
    | word arg_list redir_list
    ;

/* [ ... ]; [[ ... ]] - нельзя */
test_command:
    LBRACK arg_list RBRACK redir_list
    | LBRACK LBRACK {
        error_line(yylineno, "[[ ... ]] is not supported");
        exit(1);
    }
    ;

/* составная команда и редиректы */
compound_command:
    compound_head redir_list
    ;

assign_part:
    ASSIGN
    | ASSIGN LPAREN arg_list RPAREN
    ;

assign_list:
```

```
    assign_part
    | assign_list assign_part
    ;

arg_list:
    | arg_list arg
    ;

arg:
    word
    | assign_part
    | LBRACE WORD RBRACE
    ;

compound_head:
    if_clause
    | while_clause
    | until_clause
    | for_clause
    | select_clause
    | case_clause
    | brace_group
    | subshell
    ;

/* ( команды ) */
subshell:
    LPAREN list RPAREN
    ;

word:
    ID
    | WORD
    | STRING
    | expansion
    ;

/* $имя, $1, ${имя}, $# $@ $* $$ $? */
expansion:
    DOLLAR_ID
    | DOLLAR_NUM
    | DOLLAR_SPECIAL
    | DOLLAR_BRACE
    | cmdsubst
    | ARITH
    ;

/* $(...) */
cmdsubst:
    DOLLAR_LPAREN list RPAREN
    ;

redir_list:
    | redir_list redir
    ;
```

```
/* > >> < 2> 2>&1; << нельзя here-документ */
redir:
    GT redir_target
    | DGREAT redir_target
    | LT redir_target
    | LT LT {
        error_line(yylineno, "here-document is not supported");
        exit(1);
    }
    | REDIR_ERR redir_target
    | REDIR_ERR_OUT
    ;

redir_target:
    word
    ;

if_clause:
    IF nl_opt and_or sep THEN list FI
    | IF nl_opt and_or sep THEN list elif_parts FI
    | IF nl_opt and_or sep THEN list ELSE list FI
    | IF nl_opt and_or sep THEN list elif_parts ELSE list FI
    | IF nl_opt and_or THEN {
        error_line(yylineno, "missing ';' or newline before then");
        exit(1);
    }
    ;

elif_parts:
    ELIF nl_opt and_or sep THEN list
    | elif_parts ELIF nl_opt and_or sep THEN list
    | ELIF nl_opt and_or THEN {
        error_line(yylineno, "missing ';' or newline before then");
        exit(1);
    }
    ;

while_clause:
    WHILE nl_opt and_or sep DO list DONE
    | WHILE nl_opt and_or DO {
        error_line(yylineno, "missing ';' or newline before do");
        exit(1);
    }
    ;

until_clause:
    UNTIL nl_opt and_or sep DO list DONE
    | UNTIL nl_opt and_or DO {
        error_line(yylineno, "missing ';' or newline before do");
        exit(1);
    }
    ;

for_clause:
    FOR ID IN arg_list sep DO list DONE
    | FOR ID sep DO list DONE
```

```
    | FOR ID DO list DONE
    | FOR LPAREN LPAREN for_c_style RPAREN RPAREN sep DO list DONE
    | FOR LPAREN LPAREN for_c_style RPAREN RPAREN DO list DONE
    | FOR ID IN arg_list DO {
        error_line(yylineno, "missing ';' or newline before do");
        exit(1);
    }
    ;

/* i=0; i<10; i++ */
for_c_style:
    | for_c_style for_part
    ;

for_part:
    ID
    | WORD
    | ASSIGN
    | SEMI
    | DSEMI
    | LT
    | GT
    ;

select_clause:
    SELECT ID IN arg_list sep DO list DONE
    | SELECT ID sep DO list DONE
    | SELECT ID DO list DONE
    | SELECT ID IN arg_list DO {
        error_line(yylineno, "missing ';' or newline before do");
        exit(1);
    }
    ;

sep:
    SEMI nl_opt
    | NL nl_opt
    ;

case_clause:
    CASE arg IN nl_opt case_items ESAC
    | CASE arg ESAC {
        error_line(yylineno, "case requires in");
        exit(1);
    }
    | CASE arg SEMI {
        error_line(yylineno, "case requires in");
        exit(1);
    }
    | CASE arg NL {
        error_line(yylineno, "case requires in");
        exit(1);
    }
    ;

case_items:
```

```
    | case_items case_item
    ;

/* a|b|c) echo x ;; */
case_item:
    patterns RPAREN list DSEMI nl_opt
    | patterns RPAREN list ESAC {
        error_line(yylineno, "case item must end with ;;");
        exit(1);
    }
    ;

/* a|b|c */
patterns:
    pattern
    | patterns PIPE pattern
    ;

pattern:
    ID
    | WORD
    | STRING
    ;

/* name(), function name, function name() */
function_def:
    ID LPAREN RPAREN nl_opt brace_group
    | FUNCTION ID nl_opt brace_group
    | FUNCTION ID LPAREN RPAREN nl_opt brace_group
    ;

/* { }; перед } нужен ; или перевод строки */
brace_group:
    LBRACE nl_opt RBRACE
    | LBRACE nl_opt stmt_seq separator RBRACE
    | LBRACE nl_opt stmt_seq RBRACE {
        error_line(yylineno, "missing ';' or newline before '}'");
        exit(1);
    }
    ;

/* then или do не на месте */
stray_keyword:
    THEN {
        error_line(yylineno, "unexpected then");
        exit(1);
    }
    | DO {
        error_line(yylineno, "unexpected do");
        exit(1);
    }
    ;

/* local */
local_stmt:
    LOCAL local_names
```

```
    ;

local_names:
    local_item
    | local_names local_item
    ;

local_item:
    ID
    | WORD
    | ASSIGN
    | ASSIGN local_value
    | ASSIGN LPAREN arg_list RPAREN
    ;

local_value:
    STRING
    | expansion
    ;
%%

int main(int argc, char **argv)
{
    int result;

    if (argc > 1) {
        yyin = fopen(argv[1], "r");
        if (!yyin) {
            perror(argv[1]);
            return 1;
        }
    }
    result = yyparse();
    if (argc > 1)
        fclose(yyin);
    return result;
}
```